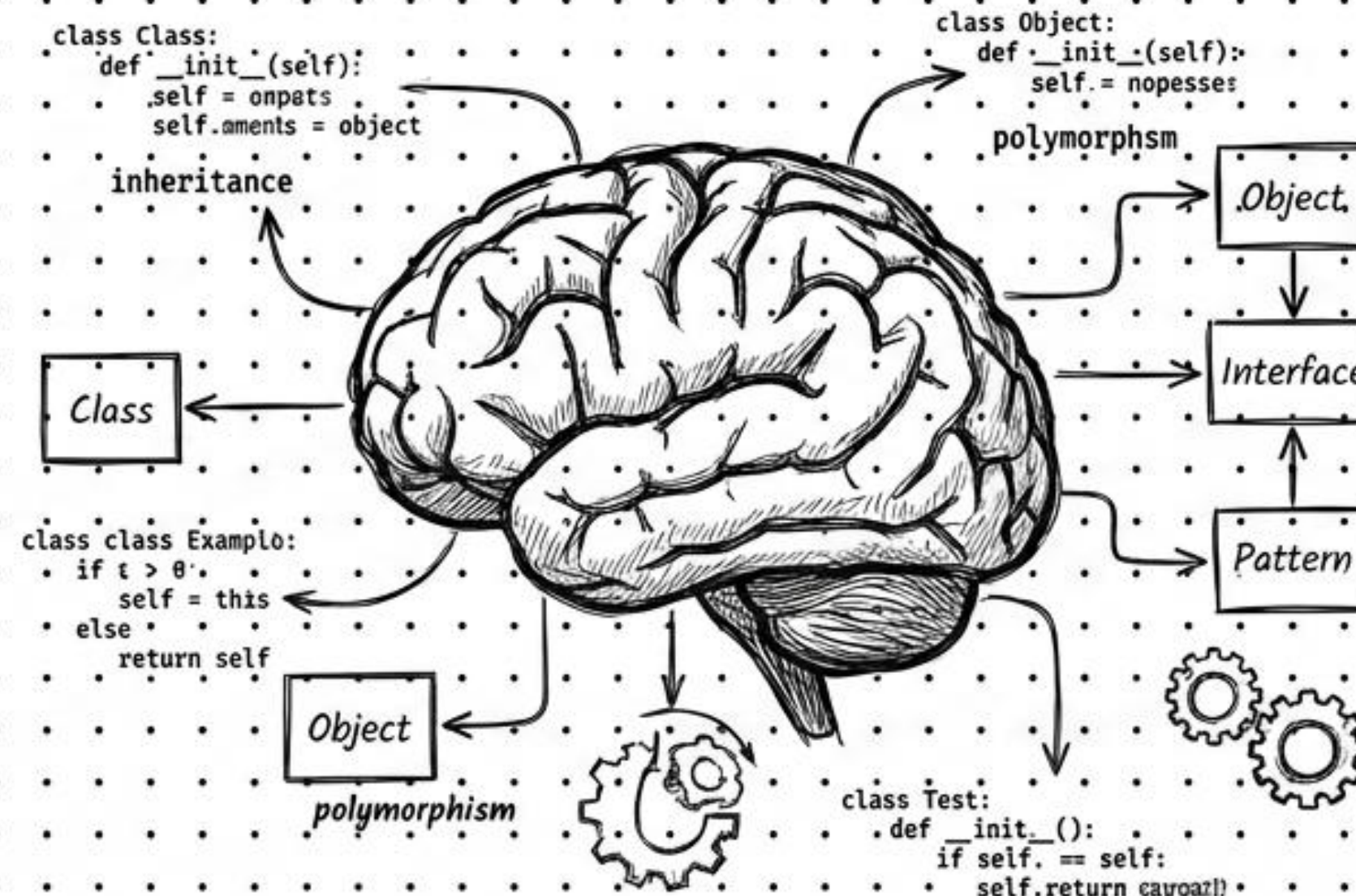


Módulo 5

Paixão por Desenvolver Talentos
Orientação a Objetos

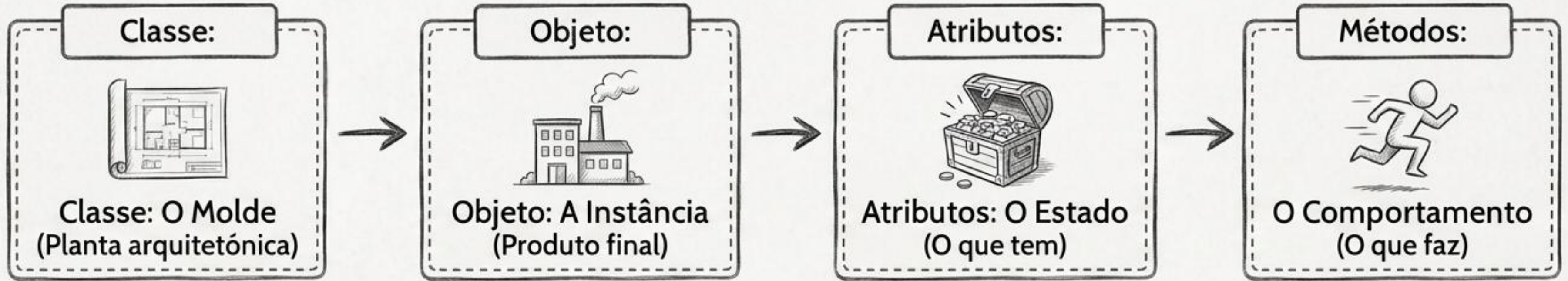
Mergulho em Programação Orientada a Objetos com Python

Arquitetura, Padrões e Design de Software



Carga Horária:
10 Horas
(2 Aulas)

A Fundação: As Peças do Universo POO



Exemplo 1: Gato

Atributos: cor, idade
Métodos: miar(), dormir()

Tom
macho, marrom



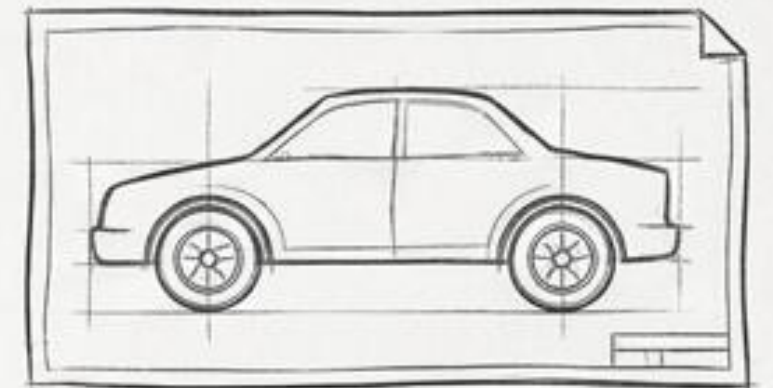
Nina
fêmea, cinza

Exemplo 2: Carro

Atributos: marca, motor
Métodos: acelerar(), frear()



Esportivo

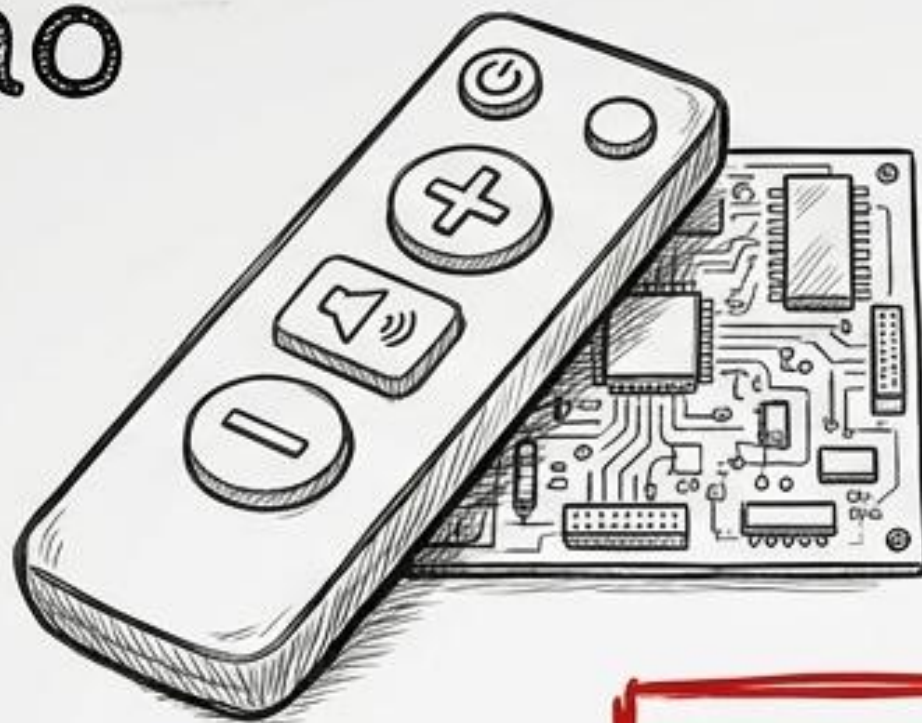


Molde de Carro



Abstração

Focar apenas no essencial.



Herança

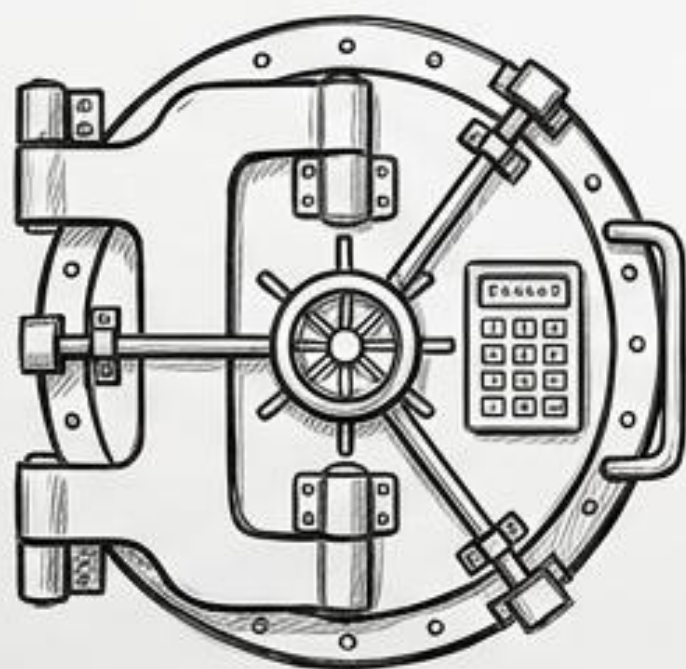
Reutilizar código e hierarquia.



Os 4 Pilares da POO

Encapsulamento

Proteger e esconder o estado.



Polimorfismo

Múltiplas formas, mesma ação.

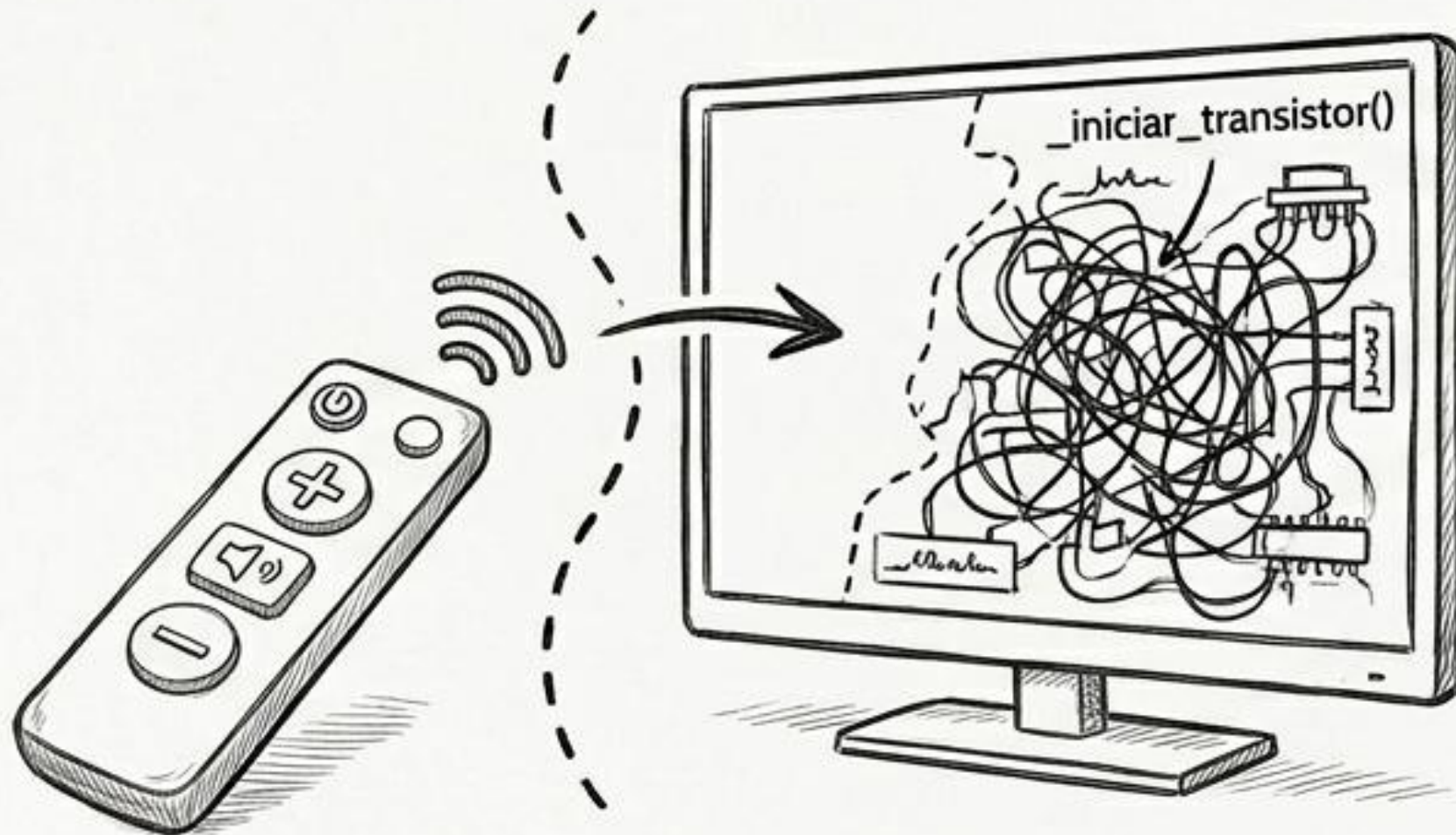


Aprofundando: Ocultando a Complexidade

Abstração A Interface Limpa

Exterior

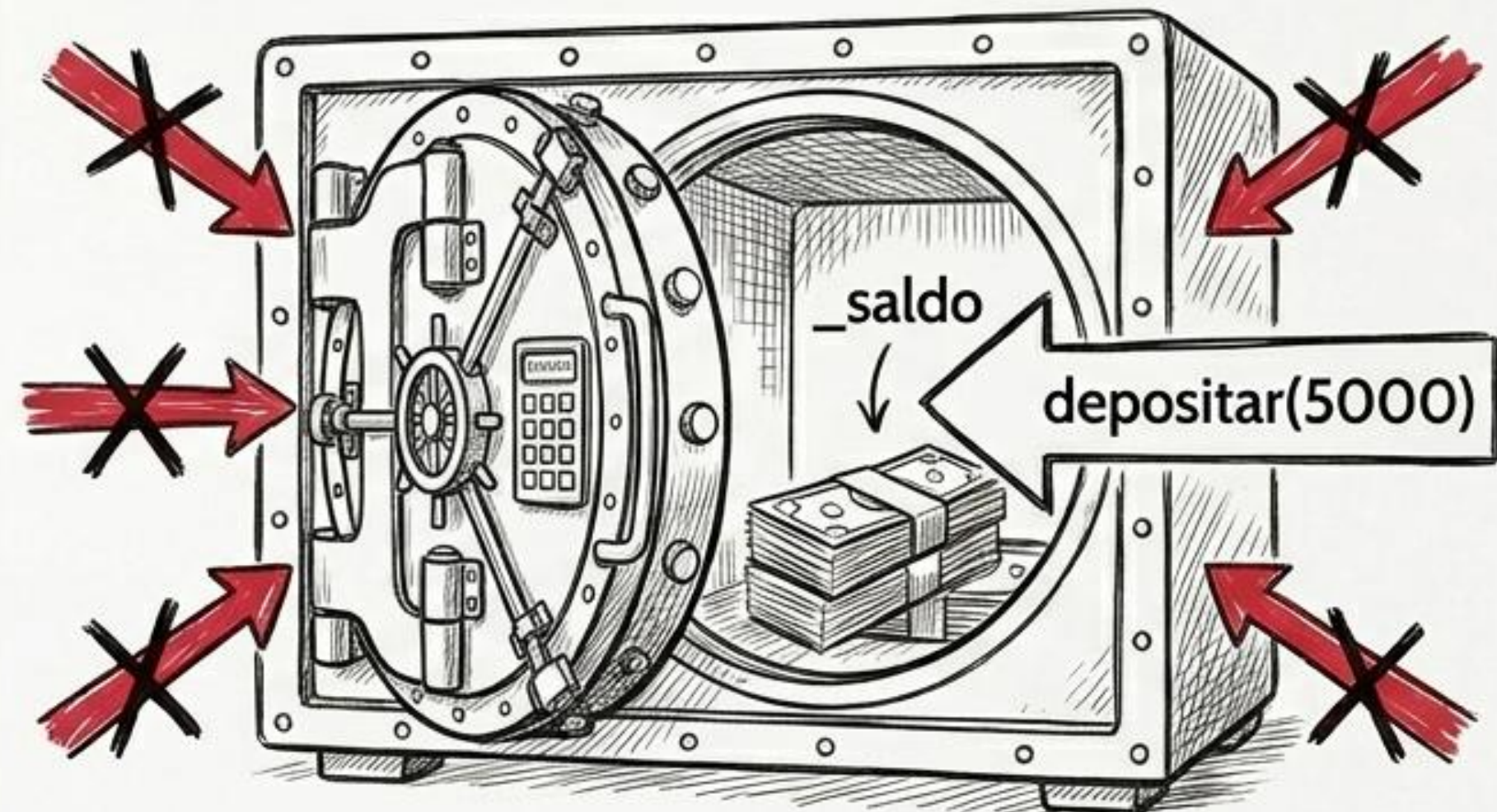
O utilizador chama `ligar()`. A voltagem e a lógica de transístores interna são ignoradas.



Encapsulamento A Proteção dos Dados

Interior

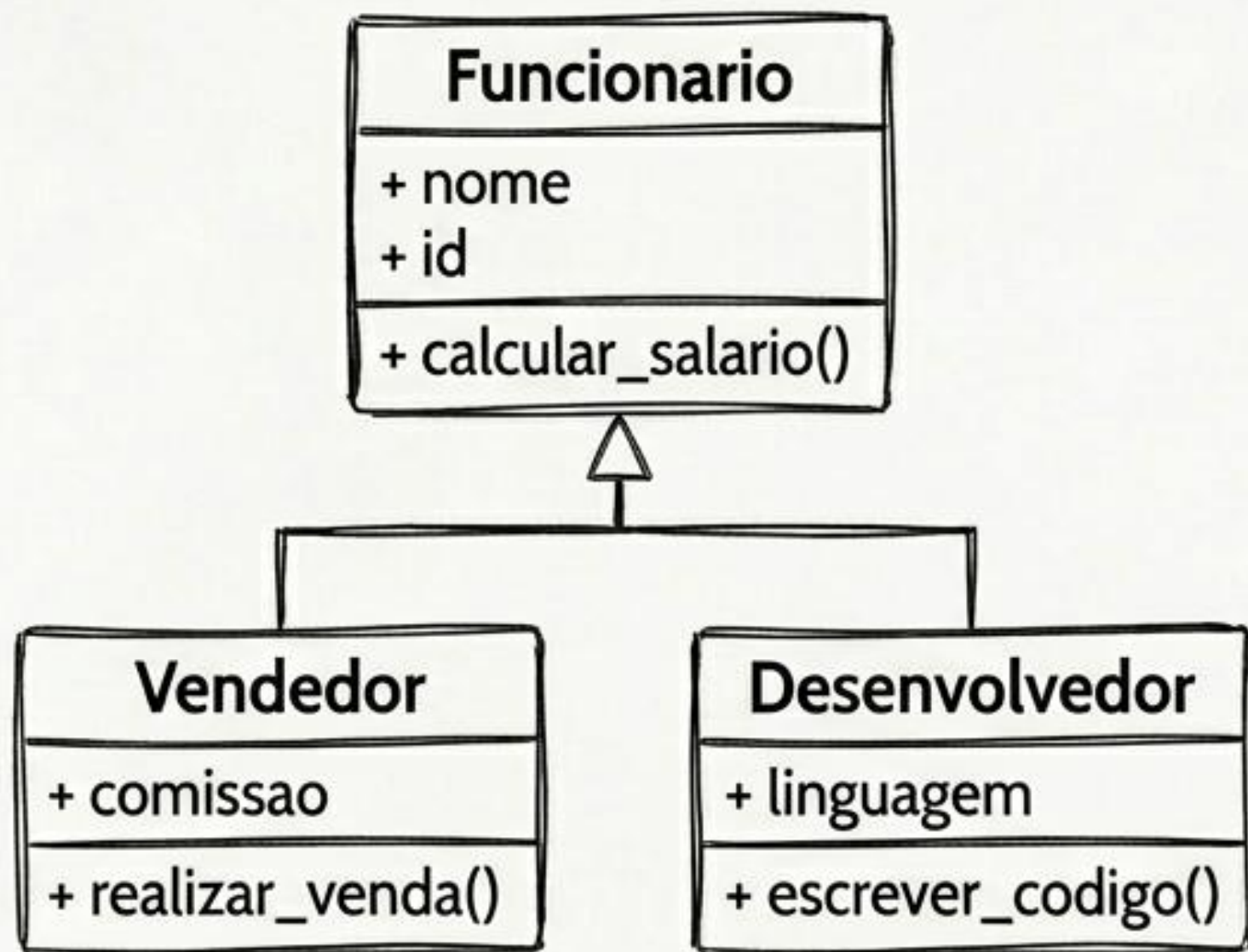
Modificadores de acesso: O saldo (`_saldo`) nunca é alterado diretamente, apenas via `depositar()`.



Aprofundando: Contratos e Elasticidade

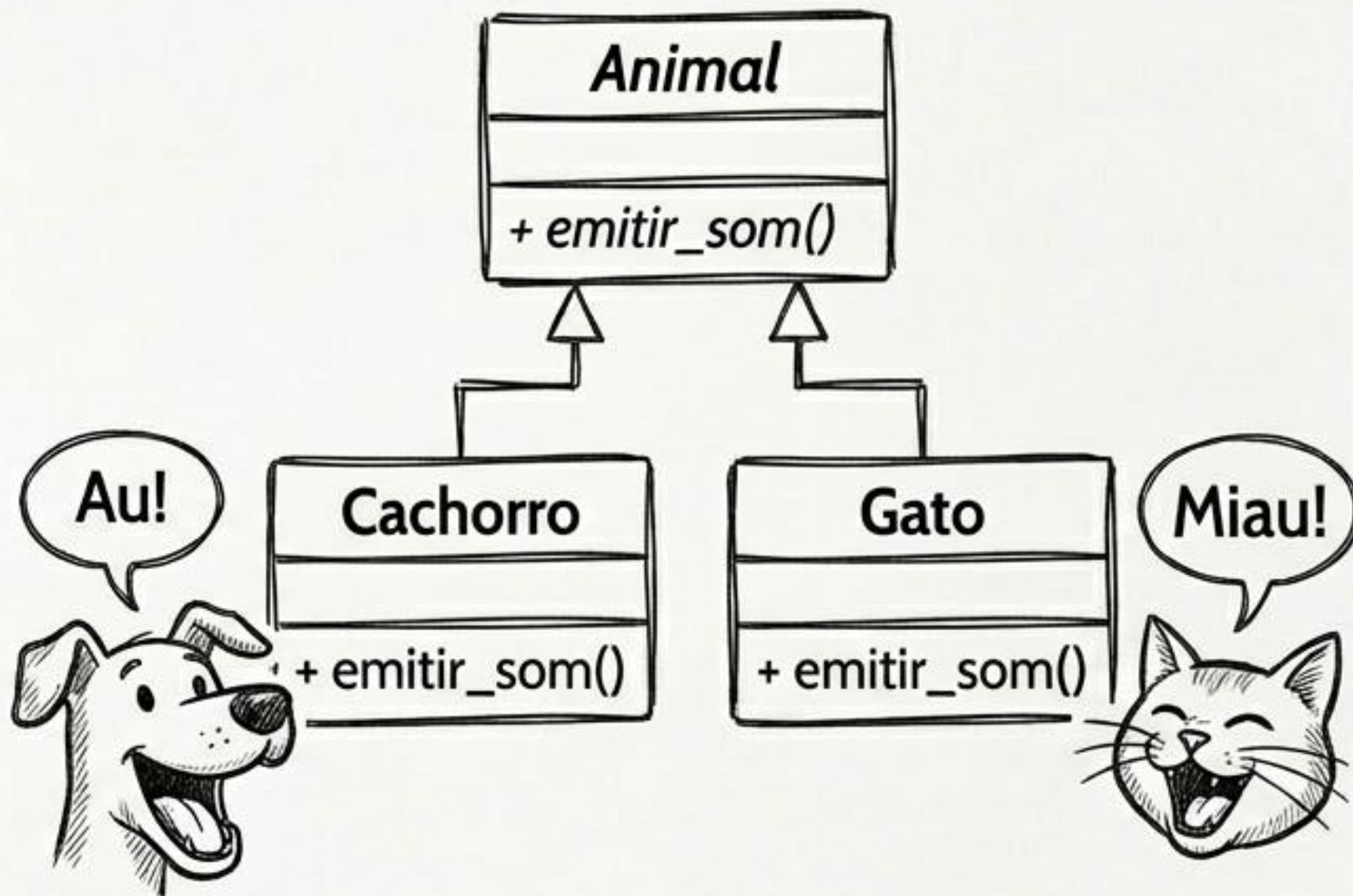
Herança

Reutilização de Código: Vendedor e Desenvolvedor herdam atributos de Funcionario, mas adicionam comportamentos únicos.



Polimorfismo

O mesmo comando, reações diferentes.



A Mágica por Trás dos Panos: Dunder Methods

Métodos com duplo sublinhado (`__metodo__`) são chamados automaticamente pelo Python.

O Nascimento



`def __init__(self):`

O Construtor.
Inicializa o estado do objeto assim que ele é instanciado.

A Vida Ativa



`def __str__(self):`

O Representador.
Define como o objeto é lido por humanos em prints.

A Morte



`def __del__(self):`

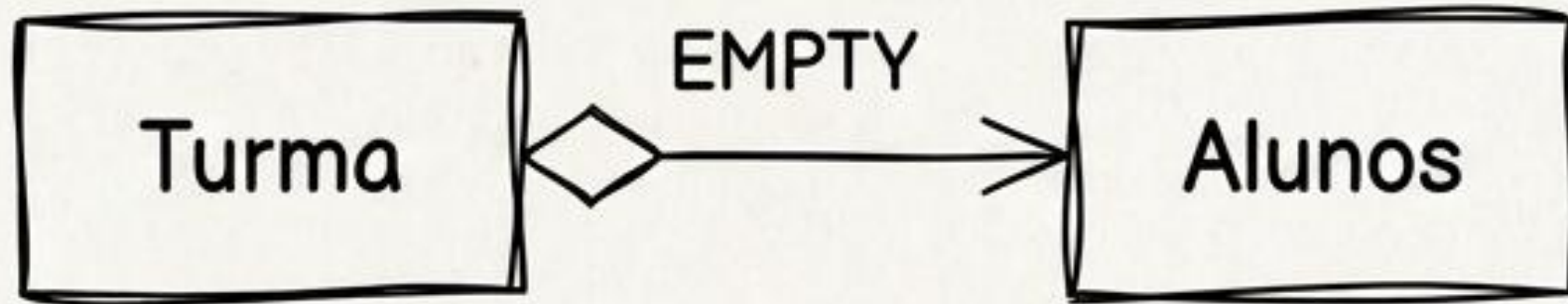
O Destrutor.
Limpa a memória quando o objeto não é mais necessário.

Relacionamentos: Composição vs. Agregação

Nem tudo é Herança ('É um'). Muitas vezes usamos 'Tem um'.

Agregação (Relação Fraca)

Turma tem Alunos. Se a turma for cancelada, os alunos continuam a existir.



Composição (Relação Forte)

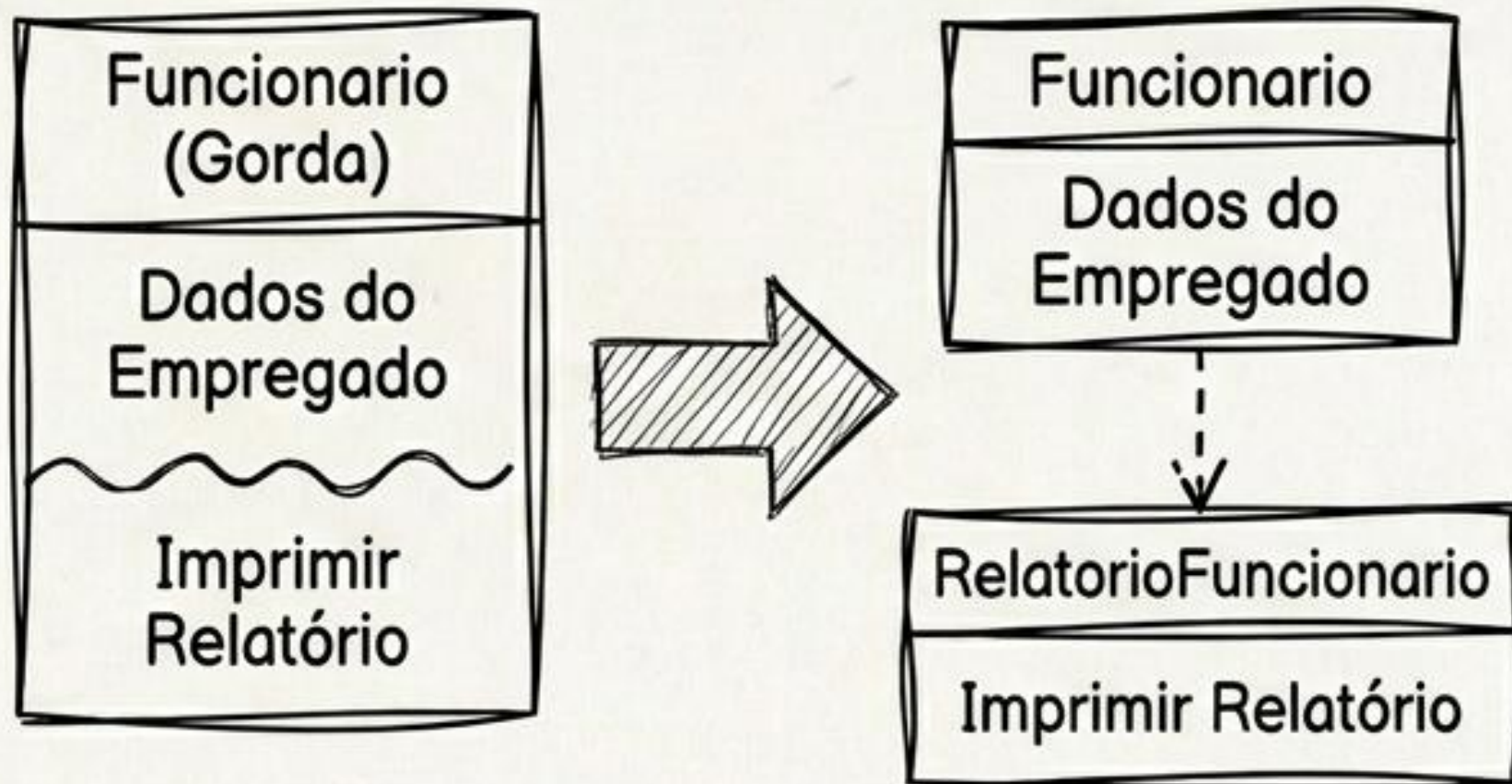
Carro tem Motor. Se o carro for destruído, o motor perde o contexto e o ciclo de vida encerra.



Design de Classes: Princípios SÓLIDOS Básicos

Responsabilidade Única (SRP)

Uma classe deve ter apenas um motivo para mudar.

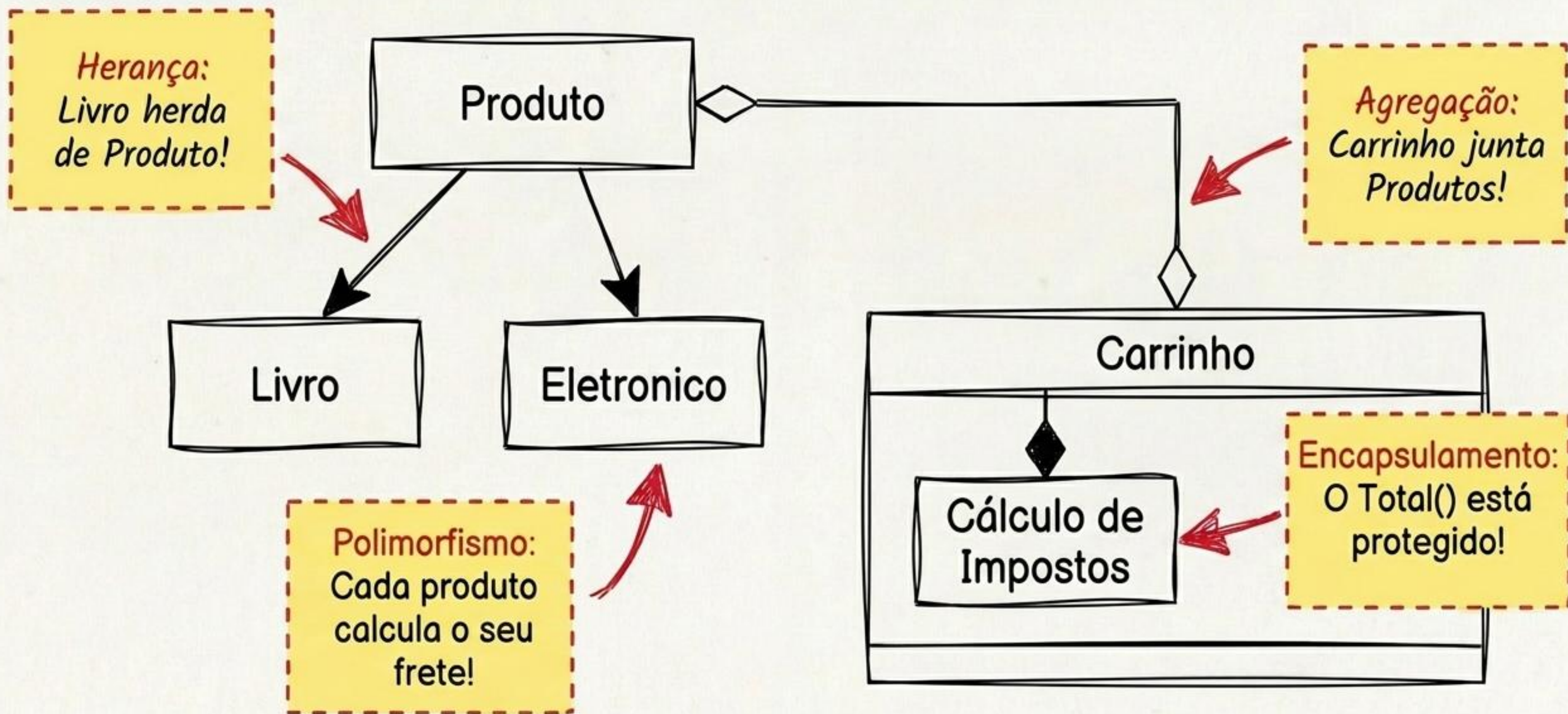


Aberto / Fechado (OCP)

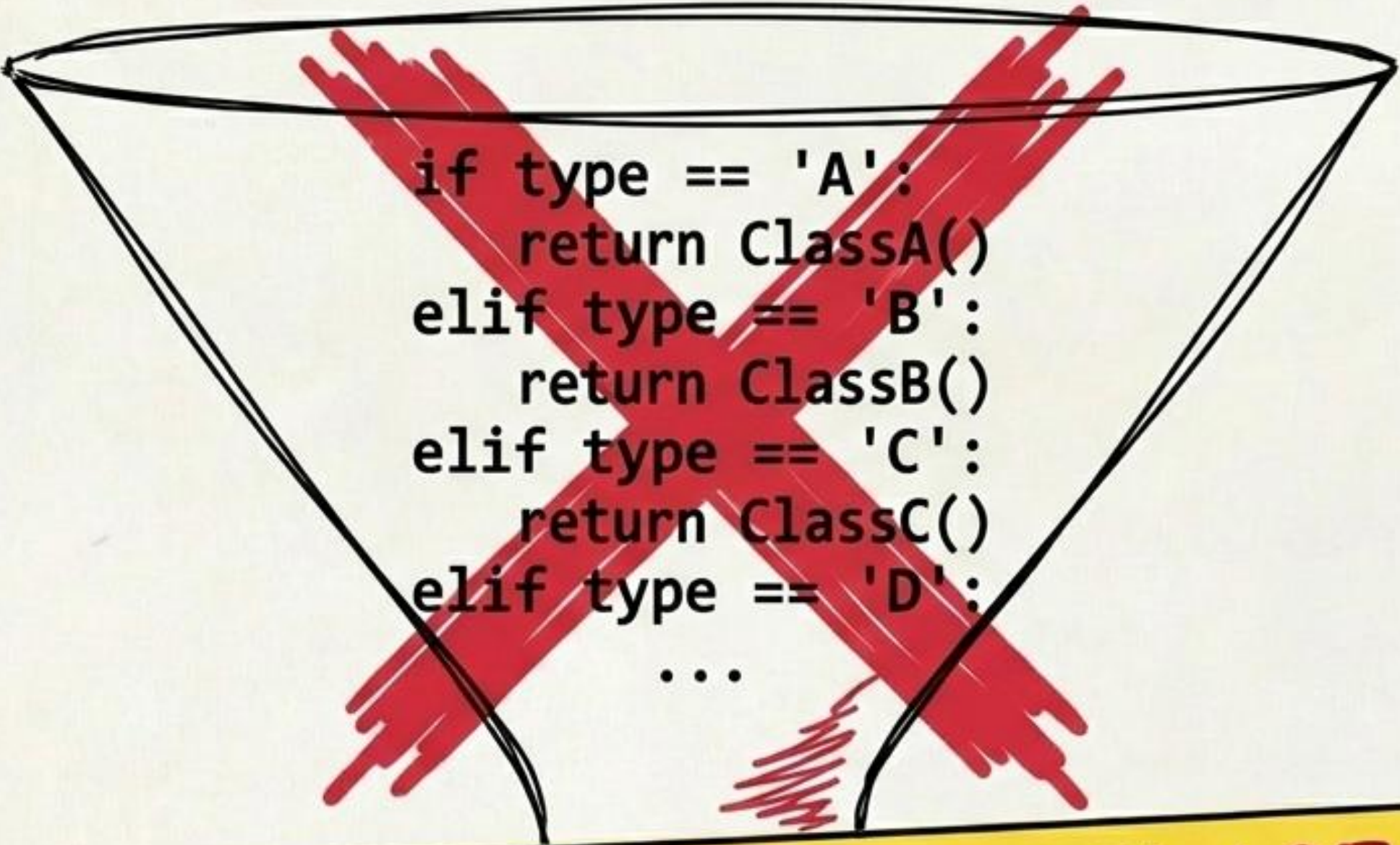
Aberto para extensão, fechado para modificação.

Use Interfaces (Polimorfismo) para adicionar novos Métodos de Pagamento, sem alterar a classe original do Carrinho de Compras.

Síntese Visual: Anatomia de um Sistema POO



O Gargalo Arquitetural: O Monstro do if/else



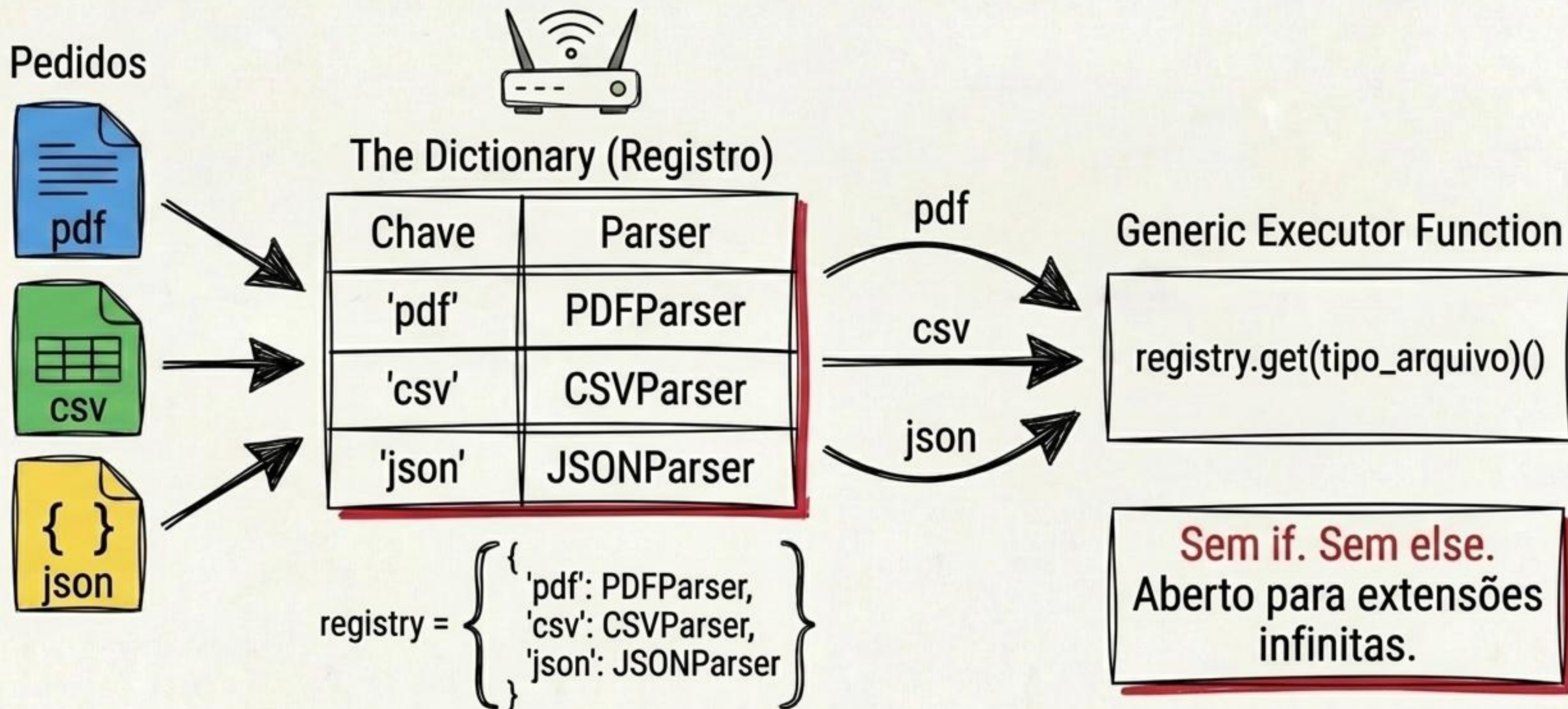
```
if type == 'A':  
    return ClassA()  
elif type == 'B':  
    return ClassB()  
elif type == 'C':  
    return ClassC()  
elif type == 'D':  
    ...
```

CODE SMELL: ALTA ACOPLAGEM

Viola o Princípio Aberto/Fechado.
Cada novo tipo exige a reescrita deste ficheiro central!

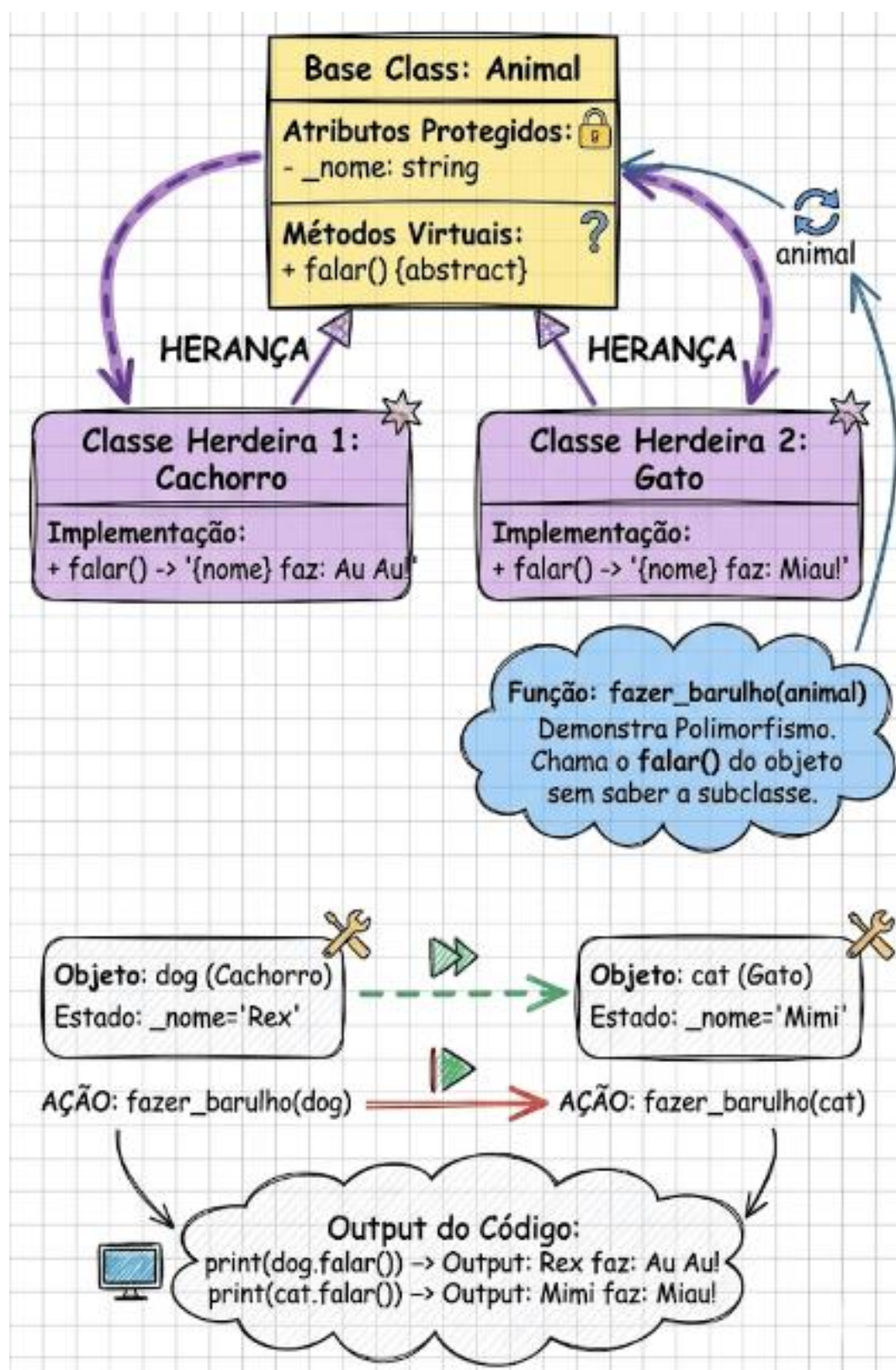
A Solução: Padrão Registry (Registro de Protótipos)

O roteamento dinâmico substitui a lógica estática (hardcoded).



Desafio Prático: Substituindo o Controlador





```
1 # Classe base (pai)
2 class Animal:
3     def __init__(self, nome):
4         self._nome = nome # Atributo protegido por convenção
5
6     def falar(self):
7         # Método que será sobrescrito pelas subclasses
8         raise NotImplementedError("Subclasse deve implementar este método")
9
10 # Classe filha que herda de Animal
11 class Cachorro(Animal):
12     def falar(self):
13         # Polimorfismo: implementação específica para cachorro
14         return f"{self._nome} faz: Au Au!"
15
16 # Outra classe filha
17 class Gato(Animal):
18     def falar(self):
19         # Polimorfismo: implementação específica para gato
20         return f"{self._nome} faz: Miaul!"
21
22 # Função que demonstra polimorfismo
23 def fazer_barulho(animal):
24     # Aceita qualquer objeto que tenha o método falar()
25     print(animal.falar())
26
27 # Instanciando e testando
28 dog = Cachorro("Rex")
29 cat = Gato("Mimi")
30
31 fazer_barulho(dog) # Saída: Rex faz: Au Au!
32 fazer_barulho(cat) # Saída: Mimi faz: Miaul!
```


Métodos especiais

Método	Função
<u><code>__init__(self, ...)</code></u>	Construtor: inicializa o objeto
<u><code>__del__(self)</code></u>	Destrutor: chamado antes da coleta de lixo
<u><code>__str__(self)</code></u>	Retorna <u>string</u> legível para <u><code>print()</code></u>
<u><code>__repr__(self)</code></u>	Retorna representação oficial para depuração
<u><code>__eq__(self, other)</code></u>	Define comportamento do operador <code>==</code>

#python

```
1 class Produto:
2     def __init__(self, nome, preco):
3         self.nome = nome
4         self.preco = preco
5
6     # Método especial para representação em string (print)
7     def __str__(self):
8         return f"Produto: {self.nome} | Preço: R${self.preco:.2f}"
9
10    # Método especial para comparação de igualdade
11    def __eq__(self, outro):
12        # Compara se nome e preço são iguais
13        return self.nome == outro.nome and self.preco == outro.preco
14
15    # Método especial chamado quando o objeto é destruído
16    def __del__(self):
17        print(f"Objeto {self.nome} foi removido da memória.")
18
19    # Criando objetos
20    p1 = Produto("Notebook", 3500.00)
21    p2 = Produto("Notebook", 3500.00)
22    p3 = Produto("Mouse", 50.00)
23
24    # Utilizando métodos especiais implicitamente
25    print(p1) # Chama __str__: Produto: Notebook | Preço: R$3500.00
26    print(p1 == p2) # Chama __eq__: True
27    print(p1 == p3) # Chama __eq__: False
```


Exercícios de Fixação em Sala (Parte 1)



1. Defina com as suas palavras a diferença fundamental entre uma Classe e um Objeto.
.....
2. Como o pilar do Encapsulamento protege e melhora a segurança do seu código em produção?
.....
3. Escreva um exemplo mínimo de Herança usando a sintaxe Python (ex: class Filha(Pai):).
.....
4. O que o Polimorfismo permite que você faça numa iteração contendo objetos heterogêneos?
.....
5. Para que serve o método mágico `__init__` e em que momento exato ele é invocado?
.....

Exercícios de Fixação em Sala (Parte 2)



6. Qual a diferença conceitual e de ciclo de vida entre Composição e Agregação?

.....

7. Cite um problema grave gerado por uma classe que viola o Princípio de Responsabilidade Única (SRP).

8. Como a estrutura de Dicionário (dict) em Python ajuda a implementar o Padrão Registry?

.....

9. Escreva o método `__str__` para uma classe Livro que retorne a string 'Título por Autor'.

.....

10. Por que utilizar múltiplos 'if/else' encadeados para instanciar classes é considerado um 'Code Smell' na ótica do Princípio OCP?

iteam | INSTITUTO
TECNOLOGICO
EDUCACIONAL
DA AMAZÔNIA

Cabô?!

Paixão por Desenvolver Talentos



Vamos implementar uma
mensageria em Python?